Pure Virtual Function and Abstract Classes

1. Pure Virtual Function

A **pure virtual function** is a virtual function that has **no required implementation in the base class** and makes the class abstract.

Syntax:

```
class Animal {
public:
    virtual void speak() = 0;
};
```

The `= 0` is what makes it **pure virtual**.

Think:

> "Every derived class MUST provide its own implementation of this function."

---

2. Abstract Class

A class containing **at least one pure virtual function** is called an **abstract class**.

```
class Animal {
public:
    virtual void speak() = 0;
};
```

`Animal` is abstract.

Therefore:

```
Animal a;  // ❌ Error
```

You cannot create an object of an abstract class.

But you **can** create a pointer/reference:

```
Animal* a;
Animal& ref = dog;
```

This is extremely important for runtime polymorphism.

---

3. Why do we need abstract classes?

Suppose you have:

```
class Animal {
public:
    virtual void speak() = 0;
};
```

There is no meaningful generic implementation of:

```
speak()
```

Every animal should define it differently.

```
class Dog : public Animal {
public:
    void speak() override {
        cout << "Bark";
    }
};

class Cat : public Animal {
public:
    void speak() override {
        cout << "Meow";
    }
};
```

Now:

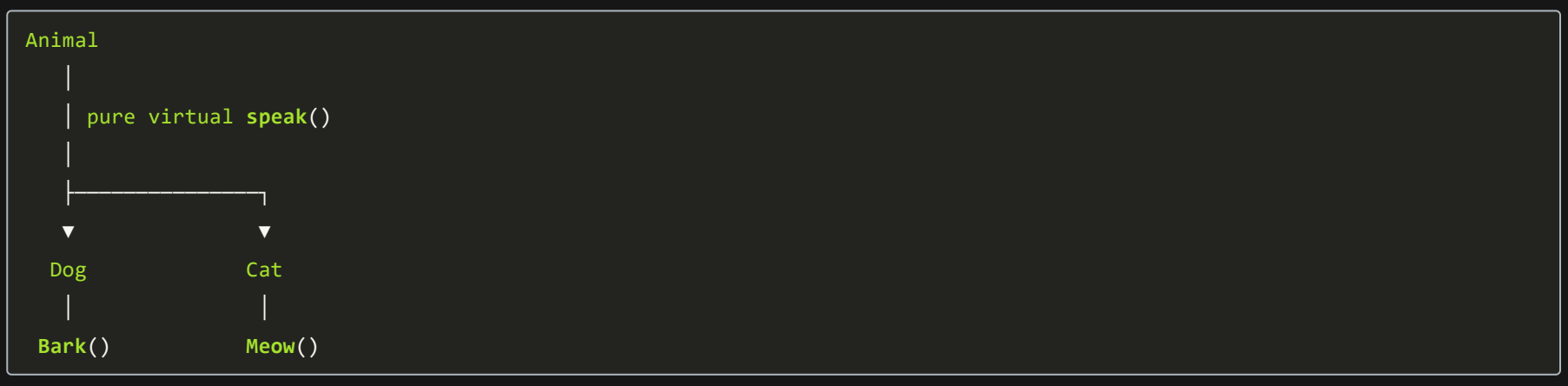
```
Animal* a1 = new Dog();
Animal* a2 = new Cat();

a1->speak();  // Bark
a2->speak();  // Meow
```

The base class defines the **interface**, while derived classes provide the **implementation**.

---

4. The key relationship

Think of it like this:



`Animal` says:

> "Any Animal must know how to speak."

But it doesn't say **how**.

That's the responsibility of `Dog`, `Cat`, etc.

---

5. What if Derived doesn't implement it?

Very important interview question.

```
class Animal {
public:
    virtual void speak() = 0;
};

class Dog : public Animal {
    // speak() not implemented
};
```

Then `Dog` is also **abstract**.

Therefore:

```
Dog d;  // ❌
```

To become concrete, Dog must implement:

```
class Dog : public Animal {
public:
    void speak() override {
        cout << "Bark";
    }
};
```

---

6. Pure virtual ≠ necessarily "no implementation"

This is a **tricky interview point**.

Many beginners think:

> Pure virtual function cannot have a body.

That's not completely true.

You can actually define a pure virtual function:

```
class Animal {
public:
    virtual void speak() = 0;
};

void Animal::speak() {
    cout << "Animal speaks";
}
```

The class is **still abstract** because `speak()` is declared pure.

A derived class can explicitly call that implementation:

```
class Dog : public Animal {
public:
    void speak() override {
        Animal::speak();
        cout << "Dog";
    }
};
```

Output:

```
Animal speaks
Dog
```

Interview answer

> A pure virtual function can have a definition, but the class remains abstract because the function is still declared with `= 0`.

---

7. Pure virtual destructor — special case

A destructor can also be pure virtual:

```
class Base {
public:
    virtual ~Base() = 0;
};
```

But there's an important catch:

A pure virtual destructor must still have a definition.

```
Base::~Base() {
}
```

Why?

Because when a derived object is destroyed:

```
Derived destructor
      ↓
Base destructor
```

the base destructor must eventually execute.

---

8. Abstract class can have normal functions

An abstract class isn't required to contain only pure virtual functions.

Example:

```
class Animal {
public:
    virtual void speak() = 0;

    void eat() {
        cout << "Eating";
    }
};
```

It can have:

- normal member functions
- virtual functions
- pure virtual functions
- constructors

- destructors
- data members
- static members

The only restriction is:

> You cannot directly instantiate an abstract class.

---

9. Can an abstract class have a constructor?

Yes.

This is another common interview question.

```
class Animal {
public:
    Animal() {
        cout << "Animal constructor";
    }

    virtual void speak() = 0;
};
```

You can't do:

```
Animal a;  // ❌
```

but when creating:

```
Dog d;
```

the `Animal` constructor still executes.

```
Dog object creation

Animal constructor
        ↓
Dog constructor
```

Why?

Because the base portion of the derived object still needs to be initialized.

---

10. Interface in C++

C++ doesn't have a dedicated `interface` keyword like Java.

Instead, you typically create an interface-like class using pure virtual functions.

Example:

```
class Drawable {
public:
    virtual void draw() = 0;
    virtual void resize() = 0;

    virtual ~Drawable() = default;
};
```

Then:

```
class Circle : public Drawable {
public:
    void draw() override {
        cout << "Drawing circle";
    }

    void resize() override {
        cout << "Resizing circle";
    }
};
```

This is commonly called an **interface class** or **abstract interface**.

---

11. Abstract class vs Interface

For interviews, understand this distinction:

Abstract class

Can contain:

```
pure virtual functions
normal functions
data members
constructors
destructor
```

Example:

```
class Animal {
public:
    virtual void speak() = 0;

    void eat() {
        cout << "eat";
    }

protected:
    int age;
};
```

Interface-like class

Usually contains mostly/all pure virtual functions and has little/no state:

```
class Printable {
public:
    virtual void print() = 0;
    virtual ~Printable() = default;
};
```

C++ doesn't enforce a formal interface construct.

---

12. Why use an abstract class?

The major reason:

> **To define a common contract/interface for derived classes.**

For example:

```
class Payment {
public:
    virtual void pay(double amount) = 0;

    virtual ~Payment() = default;
};
```

Different implementations:

```
class CreditCard : public Payment {
public:
    void pay(double amount) override {
        cout << "Credit card payment";
    }
};

class UPI : public Payment {
public:
    void pay(double amount) override {
        cout << "UPI payment";
    }
};
```

Now your code can work with the abstraction:

```
void processPayment(Payment& payment) {
    payment.pay(1000);
}
```

Then:

```
CreditCard card;
UPI upi;

processPayment(card);
processPayment(upi);
```

Same interface:

```
payment.pay()
```

Different behavior.

This is **runtime polymorphism + abstraction**.

---

13. Pure virtual vs normal virtual

This distinction is important.

Normal virtual

```
virtual void speak() {
    cout << "Animal";
}
```

Base provides an implementation.

Derived **may** override it.

```
Override optional
```

Pure virtual

```
virtual void speak() = 0;
```

Base declares a pure virtual interface.

Concrete derived classes **must** override it.

```
Override required
```

---

14. Abstract class vs Concrete class

Abstract

```
class Animal {
public:
    virtual void speak() = 0;
};
```

Cannot instantiate:

```
Animal a;  // ❌
```

Concrete

```
class Dog : public Animal {
public:
    void speak() override {
        cout << "Bark";
    }
};
```

Can instantiate:

```
Dog d;  // ☑
```

---

15. Connection to vtable/vptr

Since you just learned vtables, here's the connection.

Suppose:

```
class Animal {
public:
    virtual void speak() = 0;
};
```

Conceptually, the `Animal` vtable has an entry corresponding to `speak`, but the exact representation of a pure-virtual entry is **implementation-dependent**.

Then:

```
class Dog : public Animal {
public:
    void speak() override {
        cout << "Bark";
    }
};
```

Dog's vtable has:

```
Dog vtable
┌──────────────────┐
| Dog::speak()     |
└──────────────────┘
```

So:

```
Animal* a = new Dog();
a->speak();
```

still uses dynamic dispatch:

```
Animal*
   ↓
Dog object
   ↓
vptr
   ↓
Dog vtable
   ↓
Dog::speak()
```

Interview Questions

Q1. What is a pure virtual function?

> A virtual function declared using `= 0`. It is used to define an interface that derived classes are expected to implement.

Q2. What is an abstract class?

> A class containing at least one pure virtual function. It cannot be directly instantiated.

Q3. Can we create a pointer to an abstract class?

Yes.

```
Animal* a;
```

And:

```
Animal* a = new Dog();
```

is perfectly valid.

Q4. Can an abstract class have a constructor?

Yes.

Q5. Can an abstract class have data members?

Yes.

Q6. Can a pure virtual function have a body?

Yes.

```
virtual void foo() = 0;

void Base::foo() {
    // implementation
}
```

But the class remains abstract.

Q7. What happens if a derived class doesn't override a pure virtual function?

The derived class also becomes abstract.

Q8. Can a constructor be pure virtual?

No.

Constructors cannot be virtual, so they cannot be pure virtual.

Q9. Can a destructor be pure virtual?

Yes, but it must have a definition.

```
class Base {
public:
    virtual ~Base() = 0;
};

Base::~Base() {}
```

Q10. Why should an interface-like class have a virtual destructor?

Because you might do:

```
Base* p = new Derived();
delete p;
```

and you want:

```
Derived destructor
       ↓
Base destructor
```

to execute correctly.

What to memorize

```
Pure virtual function
        ↓
virtual foo() = 0
        ↓
Class becomes abstract
        ↓
Cannot instantiate it
        ↓
Can create pointer/reference
        ↓
Derived class must implement it
        ↓
Otherwise Derived is also abstract
```

And the interview-ready definition:

> **A pure virtual function is a virtual function declared with `= 0`, and a class containing at least one pure virtual function becomes abstract. Abstract classes cannot be instantiated directly but can be used through pointers or references to achieve abstraction and runtime polymorphism.**